

🔑 LocaliserP — Guide de paramétrage (perso)

Doc de référence pour configurer OwnTracks + le broker MQTT et fiabiliser le suivi.

- **Carte (toi)** : `https://web-production-753e55.up.railway.app` → mot de passe = `VIEW_PASSWORD`
- **Broker MQTT** : HiveMQ Cloud (gratuit) — `1f410b576ef543d8820fb7b1ce041b3e.s1.eu.hivemq.cloud:8883`

★ CHECKLIST — configurer le téléphone d'un parent (Android)

🚀 **RACCOURCI (recommandé)** : les réglages OwnTracks (2, 3, 4 ci-dessous) sont **pénibles à trouver** dans les menus Android. Utilise plutôt le **fichier de config** `config-<parent>.otrc` : édite-le sur PC pour y mettre le mot de passe HiveMQ du parent, envoie-le sur son téléphone, **ouvre-le** → **OwnTracks l'importe** et applique **tout d'un coup** (connexion + MQTT + `cmd` + `allowRemoteLocation` + Significant + locator). Il ne reste alors que les autorisations de localisation (5) et l'anti-veille Xiaomi (56).

À dérouler dans l'ordre, sur le téléphone du parent. Détails dans les §4 et §6.

1. **Play Store** → installer OwnTracks.
2. **≡ Preferences** → **Connexion** : - Mode : **MQTT** - Host : `1f410b576ef543d8820fb7b1ce041b3e.s1.eu.hivemq.cloud` · Port : `8883` · **TLS : ON** · WebSocket : OFF - Username : `maman` (ou `papa`) · Authentication : ON · Password : le mot de passe HiveMQ de ce parent - DeviceID : `phone` · TrackerID : `M / P`
3. **≡ Preferences** → **Advanced** : activer `cmd` ET `allowRemoteLocation` (sans ça, le bouton 🔄 ne marche pas).
4. **Mode de suivi** : **Significant** (PAS Move) · **locatorDisplacement** : `500` · **locatorInterval** : `3600` → le téléphone ne publie qu'après un vrai déplacement (économie de data), et répond quand même au bouton 🔄 / à l'ouverture de la page.
5. **Autorisations : localisation** « **Toujours autoriser** » + **précise**.
6. **Anti-veille Xiaomi (56)** : Démarrage automatique ON · Économiseur batterie « Aucune restriction » · **cadenas** 🔒 sur l'appli dans les récents.
7. **Publier une 1^{re} position** : onglet Carte → ▲.
8. **(Optionnel) Zones** : onglet **Zones** → créer « Maison » (centre + rayon) pour les alertes arrivée/départ.
9. **Vérifier** sur ta carte : le parent apparaît ; teste le bouton 🔄 (téléphone au 1er plan pour le 1er test).

Après ça, **teste 48 h + un reboot** : la position doit continuer à remonter sans rouvrir l'appli. Ne configurer le 2^e téléphone qu'une fois le 1^e validé.

1. Architecture

[OwnTracks Android/iOS] ⇌ MQTT/TLS ⇌ [HiveMQ Cloud] ⇌ MQTT ⇌ [Flask sur Railway] → carte protégée

- **MQTT = voie principale** : positions en direct + bouton « position fraîche » quasi instantané (le serveur pousse un ordre `reportLocation` au téléphone).
- **HTTP (/pub) = fallback** conservé (utilisable si un jour on n'a pas de broker).

2. Le broker HiveMQ Cloud (déjà créé)

- Créé via **hivemq.com** → **Start Free** → **Cloud** → **Serverless (gratuit, sans CB)**.
- **Overview** → **Connection Details** : URL (= `MQTT_HOST`) + port `8883`.
- **Access Management** → **Credentials** : 3 identifiants, tous en « **Publish and Subscribe** » :

Username	Rôle
<code>server</code>	le backend Railway (écoute les positions + envoie les ordres)
<code>maman</code>	le téléphone de maman (publie sa position + reçoit les ordres)
<code>papa</code>	le téléphone de papa

⚠ Les téléphones doivent être en **Publish and Subscribe** (pas « Subscribe only »), sinon ils ne peuvent pas publier leur position.

3. Variables d'environnement Railway (service `web`)

```
VIEW_PASSWORD=<mot de passe pour voir la carte>
TRACKERS=maman:<mdp>,papa:<mdp>          # noms des parents (whitelist + fallback HTTP)
SECRET_KEY=<longue chaîne aléatoire>
MQTT_HOST=1f410b576ef543d8820fb7b1ce041b3e.s1.eu.hivemq.cloud
MQTT_PORT=8883
MQTT_USER=server
MQTT_PASS=<mot de passe du credential "server">
# DB_PATH=/data/positions.db              # si volume monté (persistance, cf. §7)
```

Vérifier la connexion broker dans les logs : on doit voir `[MQTT] connecté au broker ✓ – abonnement à owntracks/#`

4. OwnTracks — ANDROID (téléphones des parents)

Connexion (Preferences → Connection)

- **Mode** : `MQTT`
- **Host** : `1f410b576ef543d8820fb7b1ce041b3e.s1.eu.hivemq.cloud`
- **Port** : `8883` · **TLS** : **ON** · **WebSocket** : **OFF**
- **Username** : `maman` (ou `papa`) — sert d'auth broker **et** crée le canal `owntracks/maman/<device>`
- **Password** : le mot de passe du credential HiveMQ correspondant
- **DeviceID** : `phone` (au choix) · **TrackerID** : `M` / `P`

Commandes distantes (INDISPENSABLE pour le bouton on-demand)

Preferences → **Advanced** (ou « Reporting ») : - `cmd` (**Enable remote commands**) : **ON** ← sans ça, le bouton  ne marche pas - `allowRemoteLocation` : **ON** (répondre aux ordres `reportLocation`)

Basse consommation / économie de data (forfait 2€ ≈ peu de data)

- **Mode de suivi (monitoring)** : **Significant** (surtout PAS **Move**, qui publie en continu et vide la data)

- **locatorDisplacement** : 500 (m) → ne publie qu'après 500 m de déplacement
- **locatorInterval** : 3600 (s) → pas de « ping » périodique fréquent
- Résultat : le téléphone ne consomme de la data mobile **que si elle se déplace vraiment** (et à la maison c'est le wifi). Le bouton  et l'ouverture de la page déclenchent une position à la demande.
- ⚠ Pas de mode « 100 % à la demande » dans OwnTracks : les modes qui répondent aux demandes publient aussi sur déplacement. Signifiant + seuils hauts = l'équivalent pratique.

Anti-veille (le plus important — cf. §6 pour Xiaomi)

- Localisation « **Toujours autoriser** » + précise
- Optimisation batterie **désactivée**
- Ne pas balayer la **notification permanente** (= le service qui garde MQTT vivant en arrière-plan)

5. OwnTracks — iPHONE (ton test ; futur tél. de maman)

Mêmes réglages qu'Android (Mode MQTT, Host, Port 8883, TLS ON, Username/Password, DeviceID).

Commandes distantes → **Settings** → **From Endpoint** : - `cmd` : ON - `allowRemoteLocation` : ON

⚠ Limite iOS à connaître

Quand l'appli OwnTracks est **fermée / en arrière-plan**, iOS **coupe la connexion MQTT** → le bouton « position fraîche » **ne réveille pas** l'iPhone. Ça ne marche que **appli ouverte au premier plan**. - La **dernière position connue** reste visible (mise à jour quand la personne bouge). - Le **on-demand instantané en arrière-plan** n'est **pas possible sur iOS** (limite Apple). - **Sur Android, pas ce problème** : le service permanent garde MQTT ouvert → l'ordre passe même appli fermée.

6. Réglages Xiaomi / Redmi / Poco (MIUI / HyperOS)

Le plus agressif : sans ça, OwnTracks est tué en arrière-plan. 1. **Démarrage automatique** : Paramètres → Applications → Gérer les applications → OwnTracks → **activer**. 2. **Économiseur de batterie** : même écran → « **Aucune restriction** ». 3. **Verrouiller en mémoire** : applis récentes → glisser vers le bas sur OwnTracks → **cadenas** . 4. **Localisation** : « **Toujours autoriser** » + précise. 5. (si présent) « **Autoriser l'activité en arrière-plan** ».

Samsung : Batterie → « Sans restriction » + retirer OwnTracks des apps mises en veille. **Opvo/Vivo/Honor/Huawei** : autoriser démarrage auto + exécution en arrière-plan.

7. Le bouton « Demander une position fraîche » (on-demand)

- Clique  sur une ligne du panneau → le serveur pousse un ordre `reportLocation` via MQTT.
- Le téléphone répond par une position fraîche ; la carte l'affiche dans les ~2 s (rafraîchissement accéléré).
- **Réactivité** : quelques secondes si le téléphone est joignable. Dépend du temps d'acquisition GPS.
- **Conditions** : `cmd` + `allowRemoteLocation` activés, et (Android) service en arrière-plan vivant.
- **iOS** : marche seulement appli ouverte (cf. §5).

Persistance (recommandé)

Sans volume, la base SQLite est **remise à zéro à chaque redéploiement** → le serveur oublie le canal du téléphone jusqu'à sa prochaine publication. Pour éviter ça : Railway → service → **Volumes** → monter un volume sur `/data`, puis variable `DB_PATH=/data/positions.db`.

8. Alertes (email + Telegram)

Alertes automatiques, envoyées sur **tous les canaux configurés** en même temps : -  **Batterie faible** (< `BATTERY_ALERT` %, défaut 15). -  **Zones** : entrée/sortie d'un waypoint défini dans OwnTracks (onglet **Zones**).

Email (canal principal, multi-destinataires, idéal iPhone)

Variables Railway (exemple Gmail — créer un « mot de passe d'application » sur myaccount.google.com/apppasswords) :

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=toncompte@gmail.com
SMTP_PASS=<mot de passe d'application 16 car.>
SMTP_FROM=toncompte@gmail.com
ALERT_EMAILS=enfant1@x.com,enfant2@y.com,enfant3@z.com
```

1er mail parfois en spam → marquer « non spam » une fois.

Telegram (en plus de l'email — le plus fiable, groupe = tous les enfants)

1. **@BotFather** → `/newbot` → note le **TOKEN**.
2. Créer un **groupe** Telegram avec tous les enfants.
3. **Ajouter le bot au groupe** via le lien direct (la recherche par nom échoue souvent) : `https://t.me/<nom_du_bot>?startgroup=true` → choisir le groupe.
4. **Récupérer l'ID du groupe** (méthode qui marche malgré le mode privacy) : ouvrir dans un navigateur `https://api.telegram.org/bot<TOKEN>/getUpdates` juste après l'ajout du bot → chercher `"chat":{"id":-XXXXXXXXXX}` dans le bloc `my_chat_member`. Ce **nombre négatif** = l'ID du groupe. (Si vide : retirer puis rajouter le bot au groupe, et recharger l'URL.)
5. Variables Railway :

```
TELEGRAM_TOKEN=<token>
TELEGRAM_CHAT_ID=<id négatif du groupe>
```

⚠ Ne jamais coller le token en clair ailleurs. Pour le régénérer : @BotFather → `/revoke`, puis mettre le nouveau token dans `TELEGRAM_TOKEN` (l'ID du groupe ne change pas).

Tester

Sur la carte (connecté), bouton «  **Test** » en haut à droite → envoie une alerte de test sur tous les canaux configurés (le toast indique lesquels). État actuel :  **testé, opérationnel**.

9. Dépannage

Symptôme	Cause probable	Fix
Rien sur la carte	mauvais Host/port/TLS ou permissions « Subscribe only »	vérifier §2 et §4
Bouton  sans effet	<code>cmd</code> OFF sur le téléphone	activer <code>cmd</code> et <code>allowRemoteLocation</code>
 KO quand appli fermée (iPhone)	limite iOS	normal ; tester sur Android
Position se fige après qq heures (Android)	veille / optimisation batterie	refaire §6
Après redéploiement, canal oublié	base éphémère	monter un volume (§7)

Logs serveur en direct

```
cd d:\MesProjetsIA\LocaliserP
railway logs
```

Repères : `[MQTT] connecté au broker ✓` · `[MQTT] position reçue de <parent>` · `[MQTT] ordre reportLocation envoyé...`

10. Redéployer après une modif de code

```
cd d:\MesProjetsIA\LocaliserP
git add -A ; git commit -m "..."; git push      # auto-déploiement GitHub
# ou, pour forcer immédiatement :
railway up --detach
```